

GPU Accelerated Data Manipulation

Welcome to Nvidia's workshop on Fundamentals of Accelerated Data Science.

This workshop serves as an introduction to data science with an emphasis on speed and efficiency.

It's been designed for new and practicing data scientists that are looking to accelerate workflows and optimize results with NVIDIA GPUs.

Let's get started. Here's an overview of the workshop.

Our goal is to guide you through the key stages of the data science workflow, helping you transform raw data into actionable insights.

These stages are broadly categorized into data preparation, model, training, inference, and deployment. The learning objectives for this workshop include understanding the fundamental concepts and components of data science, exploring practical examples of accelerated data science workflows, and examining the tools and methods used to achieve acceleration in data science and discussing their broader implications.

To help you achieve these objectives, we've organized the workshop into three sections. Section 1 covers an introduction to data science and data manipulation, Section 2 focuses on graph analytics, and Section 3 covers a walkthrough of various machine learning tasks. We'll conclude with a coding assessment, which will allow you to apply what you've learned and earn a certificate for completing the workshop.

Let's begin with Section 1, where we'll explore the fundamentals of data science, discussing its significance and practical applications.

Following this, we'll delve into GPU computing and Rapids, which is a suite of libraries designed to enable GPU acceleration. This will provide you with an intuitive understanding of how GPU's accelerate data processing.

Next, we'll cover key methods for data manipulation in detail, setting the stage for the Hands On lab where you'll apply these concepts directly.

Along the way, we'll also discuss data visualization and interoperability of GPU libraries. We start with this slide that showcases the wide range of impactful use cases for data science. At its core, data science is the study of data to extract meaningful insights powering applications such as customer segmentation, personalized recommendations, fraud detection, supply chain optimization, and predictive maintenance.

It lies at the heart of modern innovation, demonstrating how data has become a vital asset for businesses. Often referred to as the oil of the modern world, data is a critical resource that, when refined and utilized, effectively drives innovation, powers businesses, and transforms industries.

Companies that harness their data can gain deeper insights, optimize operations, enhance customer experiences, and maintain a competitive edge.

In today's rapidly evolving business landscape, data science is no longer just an advantage, it's a necessity. By deeply integrating data science into operations, businesses can unlock their full potential and thrive in the digital economy.

Data science is recognized as a multidisciplinary field that blends principles and practices from mathematics, statistics, artificial intelligence, and computer engineering to analyze large volumes of data.

It's a powerful approach used to ask and answer key questions such as what happened, why did it happen, what would happen next, and what actions can be taken based on the results. Data science goes beyond traditional quantitative analysis.

One of the main take away is that it also integrates knowledge of tools, methods, and technologies to extract meaningful insights and generate value from data.

Whether it's driving smarter decisions, uncovering new opportunities, or solving complex challenges, data science empowers businesses to succeed in a data-driven world.

A data scientist is responsible for tasks such as business intelligence, data mining, business analytics, predictive analysis, reporting, and data visualization.

However, the journey typically begins with raw data, which is often not immediately usable due to various challenges. This data must first undergo processing and preparation before it can be utilized effectively downstream.

While machine learning and analytics are vital outcomes of data science, non analytical tasks are equally important to ensure meaningful outcomes.

Data preparation, like ETL, extract, transform, load, plays a foundational role in any data science workflow. While the spotlight often shines on the final analysis of predictive models, it's the quality of the input data that ultimately determines the reliability and value of the outcomes.

This part of the workflow can take more than 40% of a data scientist's time.

Proper ETL ensures that raw data from various sources are collected, cleaned, and made usable for analysis.

Think of it like building a house. Without a solid foundation, the structure is destined to fail. Similarly, poorly prepared data leads to unreliable insights, inaccurate predictions, and flawed decisions.

On the other hand, clean, well prepared data allows analysts and data scientists to focus on generating value rather than wasting time fixing errors or reconciling inconsistencies. Ultimately, robust upstream data preparation ensures data is accurate, consistent, and accessible.

This empowers businesses to trust their insights, scale analytics efforts, and make informed decisions. In the data world, the same holds true.

Garbage in, garbage out.

Investing in strong data preparation processes is the cornerstone of successful analytics. You may be surprised to learn that the biggest challenges in data science today are not centered around the analytical techniques themselves.

Many of these methods and best practices are well established and widely shared. Instead, the primary hurdles lie in data processing. We frequently hear from customers about issues such as the growing size and complexity of data, as well as slowing data processing speeds.

Additionally, users often express frustration with the fragmented software ecosystem. For example, they might rely on separate libraries for data manipulation and machine learning, only to encounter compatibility issues when one library is updated, causing workflows to break.

These challenges highlight the need for more streamlined and efficient data processing solutions. To put things into perspective, let's walk through a day in the life of a data scientist. The day often begins with downloading data from the previous night.

Ideally, the data has arrived on time, allowing the data scientist to quickly implement their first workflow. However, due to the large size of the data set, the processing can take a significant amount of time to complete.

When working with new data sets, it's common to forget a step or make a small mistake. But with a heavy data processing workload that takes hours, even minor errors can drastically slow down progress, making it difficult to move beyond data preparation and focus on analytics.

This not only disrupts productivity but also limits opportunities for exploration and experimentation. To overcome this, we need workflows that are fast, efficient, and support iterative experimentation. Here's a glimpse into the day of a data scientist leveraging and accelerating computing workflow.

With data processing no longer a bottleneck, they're able to achieve higher productivity, allowing them to focus on meaningful tasks.

They can do more exploration and iteration, which leads to improved results and deeper insights. This results in reduced time to insight and lower total cost of ownership, benefiting both the individual and the entire team.

Acceleration can be achieved by leveraging specialized hardware like the GPU. By adopting GPU acceleration, we can sustain the exponential growth and computing performance.

It's projected that by 2025, GPU computing will deliver 1000 times the performance of single threaded computing. This remarkable advancement has unlocked groundbreaking applications such as recommendation systems, large language models, and genomic analysis.

Today's enterprises rely on this immense computing power to stay ahead and solving problems like forecasting, fraud detection, and cybersecurity.

GPU acceleration is truly a driving force for modern problem solving and progress.

To understand how acceleration happens, let's take a closer look at the differences between ACPU and a GPU. The CPU is a general purpose processor responsible for managing most system tasks, including running the operating system, handling input, output operations, and executing high level sequential instructions.

It's designed to have a small number of powerful core optimized for sequential processing and low latency tasks.

In contrast, the GPU is a specialized processor built for parallel processing. It contains thousands of smaller efficient cores designed to handle tasks that can be parallelized, such as matrix operations and vector computations.

This architectural difference exists because the two processors are built with distinct goals in mind. CPUs excel at executing a few threads very quickly and efficiently, focusing on sequential tasks. GPUs excel at executing thousands of threads simultaneously, making them ideal for tasks requiring high levels of parallelism.

Consider calculating the aggregate sum of a large data set loaded into memory on a GPU. The data set is divided into smaller chunks, and thousands of threads independently calculate the partial sums in parallel.

The partial sums are then combined to produce the final result, significantly speeding up the process on ACPU with limited number of cores.

Summing the data set involves processing one or few chunks at a time in a more sequential manner. If this were the only task needed to be performed, it would be faster on the GPU and potentially using less energy.

Not all tasks benefit from the GPU execution, however. For workloads that involve high parallelism, such as video and image rendering, machine learning and data science, GPUs offer significant acceleration.

By offloading such tasks to the GPU, systems can achieve faster performance and improved efficiency. In order to perform calculations on the GPU, data has to be loaded on the GPU memory directly from the disk or copied from the CPU memory.

As we progress through the slides, we'll often bring up memory as a key consideration. Let's take a visual look at a simple example involving structured data.

Structured data is defined by having a predefined format or schema. It's often represented as tables or spreadsheets originating from sources like CSV files, Excel spreadsheets, or relational databases. Once loaded into memory, structured data is typically represented as a data frame, a format popularized by the Python pandas library for its flexibility and ease of use.

The world also contains semi structured and unstructured data. As you'll soon realize, the efficiency of data processing largely depends on its structure.

Data frames consist of columns and rows, which is why they're often referred to as tabular data.

Each column represents a feature, property, or dimension, while each row corresponds to an individual record or observation.

Within the computer's memory, this data is stored as a matrix. Suppose we need to compute the result of column one minus column 2 multiplied by column 3A.

Straightforward approach would involve processing one row at a time, performing operations on each value sequentially. While intuitive, this method requires numerous simple operations to be executed in sequence, which can be time consuming for the large data sets.

By leveraging vector operations, computations can be completed in far fewer steps. The key idea is that the same operation can be performed simultaneously across multiple elements of a vector using multiple cores.

This parallel execution significantly boosts performance as the total computational power scales with both the vector size and the number of cores available.

This is, of course, just a tip of the iceberg when it comes to the technology that makes GPUs so efficient. However, it highlights a key factor behind their acceleration capabilities, parallelism through vector operations.

There are several ways to interact with a GPU, depending on your experience and goals. At the lowest level, developers can write CUDA code to directly leverage GPU capabilities.

This approach is often preferred by experienced programmers aiming to maximize GPU's performance. For data scientists, a more accessible option is using GPU accelerator Python libraries.

These libraries often mimic the syntax of popular CPU based libraries, but internally invoke low level CUDA code to execute on the GPU.

This allows data scientists to harness GPU power without having to write CUDA code themselves.

This concept is known as an abstraction layer, which simplifies the workflow as you move up the stack. Additionally, there are no plugins that enable GPU acceleration for widely used data science libraries with no code changes required under the hood, these plug-ins seamlessly integrate with GPU optimized Python libraries.

One such example is QDF, a GPU accelerated library that provides fast, efficient data manipulation while maintaining a familiar interface similar to CPU based tools.

QDF is a GPU accelerated Python library for loading, joining, aggregating, filtering, and otherwise manipulating data using a data frame style API.

Some of the features include efficient input output, columnar memory format, rich data operations, and the ability to write user defined functions.

We've touched on how it achieves faster statistical analysis.

It also helps us generate faster responses from interactive visualizations as well as data preprocessing and feature engineering for downstream machine learning tasks.

Apache Arrow is an open source columnar in memory data format designed for high performance analytics and efficient data processing. The key advantages include columnar memory layout.

Data is stored in the columns, which perfectly aligns for parallel processing on GPUs, enabling fast and efficient vector operations.

For example, a lot of data analytics involve aggregation of values from a single column. When the data is in a columnar format, they're easily accessed without the need to read unrelated data.

0 Copy data sharing and improved interoperability, which allows data to be shared across libraries without copying or serialization, reducing overhead and improved performance and efficient input and output. Arrow accelerates reading and writing of columnar file formats such as Parquet.

A group by operation is a powerful technique that enables data scientists to split data into groups based on shared characteristics or specific criteria.

These operations can be used in various ways, including aggregation, transformation, and apply functions. They follow a split, apply, combine approach to produce a new data frame.

Group by aggregations, such as calculating common descriptive statistics like sum, mean, count and Max are considered reduction functions that execute very quickly.

However, there are certain scenarios where group by operations may not benefit from GPU acceleration. Consider a use case where the data scientist needs to calculate the difference between a value and the group mean.

This can be implemented using a group by apply operation. The apply function works by applying a specified function to each group sequentially and then concatenating the results.

However, this approach can be very slow when dealing with large number of small groups due to the overhead of processing each group individually.

On the other hand, for a small number of large groups, the performance can be acceptable. An alternative solution involves using group by transform, which can be far more efficient when carefully designed to leverage vector operations on the GPU.

Vector operations are key to maximizing performance, and they allow for faster execution compared to sequential approaches.

However, it's important to be mindful of increased memory utilization from intermediary data frames. The memory footprint can be influenced by the number of groups and the size of each group, so efficient code design is crucial for optimal performance.

We aim to bring GPU acceleration to where data scientists already work, while minimizing friction and simplifying adoption.

We've done a lot of work to accelerate existing workflows using GPU's with QDF without requiring any code modifications.

We've achieved this for Pandas, the undisputed leader for working with tabular data, which has an extensive ecosystem of libraries built around it.

Similarly, we've enabled GPU acceleration for Polar's, a rapidly growing data frame library known for its speed and efficiency in data manipulation and analysis.

Additionally, we've extended this capability to distributed computing frameworks like Dask and Spark, ensuring that large scale data processing can fully leverage GPU power.

This slide highlights the various ways you can seamlessly integrate GPU acceleration into your workflows. By enabling these popular data frame libraries, we aim to deliver the most value to data scientists and the broader analytics community.

Most data scientists are familiar with Pandas when working with tabular data.

Pandas is widely adopted because of its flexible API and strong compatibility with other libraries, making collaboration on projects easy.

Its popularity extends so far that even large language models understand it well.

You see this when LLM based chat bots answer data analysis questions using Pandas data frames.

Pandas is ideal for prototyping and exploratory analysis on small data sets.

However, it struggles to scale as data size increases.

This limitation exists because many pandas functions are single threaded, meaning they don't utilize more than one CPU core.

To help users retain the benefits of pandas while achieving significant performance improvements, we've introduced A0 code change GPU accelerator called QDF pandas.

With QDF pandas, no changes to your existing pandas code are required. You simply load QDF pandas when running your pandas code. Then the execution will prefer to use the equivalent QDF functions and methods.

If a particular operation is not supported, it seamlessly falls back to pandas on the CPU. This hybrid approach ensures compatibility and performance without disruption.

In our benchmarks, this method has shown up to 150 times performance improvements on typical data analysis workflows.

Holers is a high performance data frame library built in Rust designed for speed, scalability and efficient memory usage. It's multi threaded architecture and lazy execution model combined with modern data format support making an excellent choice for large scale data processing where performance is critical.

Polar's is a modern, faster alternative to pandas, particularly well suited for big data workloads that demand efficiency and speed.

Additionally, Polar's workflows can be GPU accelerated. Unlocking even greater performance for compute heavy tasks, one of Polar's key strings lies in its lazy execution and query optimization capabilities.

These features enable Polar's to optimize operations before execution, ensuring that computations are performed efficiently.

Let's dive deeper into what this means and how it benefits data processing workflows. In lazy execution, operations are not executed immediately.

Instead, they're deferred and combined into a logical query plan.

This approach allows the system to wait until a final action, such as collecting the result, is triggered before execution.

The query optimizer analyzes the logical plan and reorganizes operations to maximize sufficiency.

For example, in this scenario, Polar's execution engine intelligently filters the column while the CSV is being loaded instead of performing these steps separately.

This approach allows the system to analyze operations before execution, enabling it to eliminate redundant steps, reorder tasks for improved efficiency, and minimize computation and memory usage.

When using a GPU engine, the workload prioritizes execution on the GPU for accelerated performance. If an operation is not supported on the GPU, it seamlessly falls back to the CPU, ensuring smooth and reliable processing.

For data sets that don't fit into a single GPU, we need to use distributed computing to scale data processing across a cluster of computing nodes that work together as a single system.

By using libraries like DAS, we can harness the power of multiple GPUs within the same system, where each GPU acts as an individual worker.

DAS splits the data into smaller manageable chunks or partitions, even if we refer to them as a single DAS data frame. With this structure, users can leverage familiar data frame style APIs, while Dask handles the coordination across workers.

Dask also employs lazy execution, where tasks are not executed immediately.

Instead, a client builds and optimizes a task graph to improve efficiency.

A scheduler then distributes these tasks to the worker nodes for execution. This approach is a powerful solution for massive data sets, enabling scalability and performance.

However, it's important to monitor and rebalance data and workloads across workers when necessary to ensure optimal performance. In modern data science, we have a variety of tools for data manipulation and analysis, and selecting the right one depends on the size of your data, your hardware, and your performance requirements.

Pandas is ideal for small to moderately sized data sets that fit into memory. It's excellent for prototyping workflows and testing algorithms due to its simplicity and flexibility.

Polars shine when working with medium to large data sets on a single machine, especially when performance is critical.

Its multi threaded optimized design makes it a faster alternative to Pandas for large scale data sets that don't fit into memory. Tools like Dask and Spark Excel by leveraging distributed computing.

Choosing the right tool ensures you achieve optimal performance for your data processing tasks. Whether you're working on a laptop, a server, or a large computing cluster, these can all be GPU accelerated. The next thing we can do with data is data visualization.

It's the process of representing data graphically to uncover patterns, trends, and insights.

You're likely familiar with common chart types such as scatter plots, line charts, and bar charts.

These tools bridge the gap between raw data and understanding, enabling us to see a story within the data.

Visualizations not only help us communicate insights at the end of a project, but should also play a key role throughout the data acceleration and enrichment process.

They allow us to identify outliers, anomalies, and hidden patterns that might be missed using purely analytical methods.

Interactive visualizations take this further, allowing users to explore data dynamically, uncover deeper insights, and enhance engagement.

However, with large data sets, interactions can become slow and disruptive.

To reduce this friction, a GPU accelerated solution is essential.

Rapids libraries seamlessly integrate into visualization workflows, with many tools offering direct QDF support or easy integration.

For example, we can combine the following tools to build a powerful visualization engine, HV Plot to create interactive plots from QDF data frames, data shader for efficiently rendering massive data sets, Q cross Filter to build cross filtered dashboards with multi dimensional interactivity and Plotly Dash to deploy these dashboards as fully functional applications.

Together, this combination enables real time GPU accelerated visual exploration of even the largest data sets. Q Cross Filter is a Rapids framework designed to connect web visualizations to GPU accelerated cross filtering.

It enables interactive, lightning fast filtering of large tabular data sets via QDF.

By eliminating the complexity of connecting multiple charts to a GPU back end, Q Cross Filter allows you to focus on exploring data faster and with greater ease.

Data Shader is a Python library for large scale data visualization capable of rendering data sets with millions or even billions of points.

Instead of drawing every point individually, Data Shader dynamically aggregates data into pixels, ensuring it scales sufficiently regardless of data size.

The library uses a three-step pipeline to render data. Rasterization aggregates data into a pixel grid.

Transformation applies operations such as shading or normalization. Rendering produces the final visual output for even faster performance on massive data sets.

Data shader can be GPU accelerated by leveraging Rapids, QDF, or other GPU based tools during the data aggregation step. This integration significantly speeds up the rasterization process, making it a powerful solution for visualizing extremely large data sets with high efficiency.

This is an excellent opportunity to discuss the evolution of GPU accelerated data science. While GPU acceleration itself is not new, NVIDIA has progressively developed optimizations for specific processes over time.

However, these applications often required specific data formats for different steps, leading to frequent data copying and conversions throughout the pipeline.

Unfortunately, these extra operations can reduce or negate the performance gains achieved from GPU acceleration.

By expanding functionality within Rapids, we can minimize or even eliminate data transfers across systems. Ideally, all processes would utilize a shared data memory format, removing overhead caused by cross system communication and ensuring a seamless high performance workflow.

At present, the Apache Arrow columnar data specification has emerged as the preferred standard, with many GPU libraries, including those for machine learning, being built on this format.

The key advantage lies in the zero copy principle and efficient data sharing, which greatly improved performance and interoperability across systems.

Beyond what we've discussed, Rapids ensures strong interoperability with QPY for array computations using CUDA and with QDNN, which are widely adopted in most deep learning frameworks.

This means that data preprocessing and QDF can seamlessly flow into machine learning workflows with minimal overhead, enabling efficient and high performance data pipelines.

To summarize, Rapids is a suite of GPU accelerated libraries designed to speed up end to end data science workflows on the GPU.

These libraries leverage CUDA for low level computational optimization and are built to mimic existing APIs or service plug insurance for popular libraries, ensuring a seamless transition.

Rapids libraries are easy to install and deploy across different environments, whether it's on the cloud, in a data center, or on your workstation, making GPU acceleration accessible and efficient for all data science tasks. Lastly, when working with GPUs, there's several key considerations to ensure optimal performance.

Monitor memory utilization. Leverage vector operations wherever possible for faster computations. Keep data on the GPU as much as possible to minimize transfer overhead and diagnose bottlenecks to identify and resolve performance issues effectively.

By keeping these principles in mind, you can maximize the efficiency of GPU accelerator workflows. Go ahead and go through the hands on lab.